



**UNIVERSITÉ
DE GENÈVE**



**UNIVERSITÉ
DE GENÈVE**

FACULTÉ DES SCIENCES

UNIVERSITÉ DE GENÈVE

FACULTÉ DES SCIENCES

Département d'Informatique

Travail de Bachelor en Sciences Informatiques

Année académique 2025-2026

Programmation dynamique pour formules SAT

Auteur :

Elie BUSSOD

Superviseurs :

Prof. Arnaud CASTEIGTS

Prof. Pierre LEONE

Date de rendu : Juin 2026

Résumé

Bla bla abstract

Table des matières

Résumé	1
1 Introduction	4
1.1 Contexte et motivation	4
1.2 Approche structurelle et décomposition arborescente	5
1.3 Objectifs de ce travail	5
2 État de l’art	6
2.1 SAT, #SAT et MaxSAT	6
2.1.1 Le problème SAT	6
2.1.2 #SAT : comptage de modèles	6
2.1.3 MaxSAT	7
2.2 Algorithmes de résolution	7
2.2.1 DPLL et CDCL	7
2.2.2 Extensions pour #SAT et MaxSAT	7
2.2.3 SMT et compilation de connaissances	7
2.3 Décompositions arborescentes et paramètres de largeur	7
2.4 PS-width et l’algorithme de Sæther-Telle-Vatshelle	8
2.5 Compilation de connaissances CNF $\rightarrow$ d-DNNF	8
2.5.1 Langages cibles	9
2.5.2 Requêtes tractables sur d-DNNF	9
2.5.3 Compilation explicite pilotée par la ps-width : BCMS 2016	9
2.5.4 Évaluation et vérification	9
3 Cadre théorique	10
4 Implémentation	11
5 Résultats expérimentaux	12
6 Discussion	13
7 Conclusion	14
A Annexes	15
Bibliographie	16
Déclaration d’utilisation de l’IA	19

Table des figures

1

Introduction

1.1 Contexte et motivation

La satisfaisabilité propositionnelle (SAT) occupe une place centrale en informatique théorique et appliquée depuis Cook [1], qui en a établi la NP-complétude, et Karp [2], qui a tiré de cette propriété une vaste famille de réductions polynomiales et fait de SAT le problème de référence de la théorie de la complexité. Malgré cette intractabilité théorique dans le cas général, les solveurs modernes parviennent à traiter des instances industrielles comportant des millions de variables et de clauses, grâce notamment à l'algorithme d'apprentissage de clauses guidé par les conflits (CDCL) [6, 5], à la propagation unitaire (BCP) et à des heuristiques de branchement sophistiquées. Cette efficacité pratique a conduit à une adoption massive de SAT dans des domaines aussi variés que la vérification formelle de circuits électroniques, la planification automatique, la synthèse de programmes et la cryptanalyse.

Ce succès a naturellement suscité un intérêt croissant pour des variantes plus expressives : le *comptage de modèles* (#SAT) et la *maximisation de clauses satisfaites* (MaxSAT). Le problème #SAT consiste à déterminer le nombre exact d'affectations satisfaisant une formule CNF donnée. Il généralise la décision de satisfaisabilité en quantifiant la cardinalité de l'ensemble des modèles, et intervient naturellement dans le raisonnement probabiliste, l'inférence dans les réseaux bayésiens et la vérification probabiliste [10]. Le problème MaxSAT, quant à lui, cherche une affectation qui satisfait le plus grand nombre possible de clauses, même lorsque la formule est globalement insatisfaisable. Il modélise des problèmes d'optimisation combinatoire à contraintes souples (ordonnancement, configuration, bioinformatique) et constitue un cadre unifiant pour de nombreux problèmes NP-difficiles.

Ces deux variantes sont strictement plus difficiles que SAT : #SAT est #P-complet [3] et MaxSAT est NP-difficile. Cette complexité accrue justifie le développement d'approches algorithmiques spécialisées.

Une stratégie complémentaire à la résolution directe consiste à transformer hors-ligne la formule en une représentation structurée, typiquement un graphe orienté acyclique de type d-DNNF, sur laquelle un large éventail de requêtes (comptage, satisfaisabilité, équivalence) deviennent polynomiales. Cette *compilation de connaissances* [10] amortit le coût hors-ligne sur de multiples interrogations ultérieures, et est devenue le standard pour l'inférence probabiliste exacte et la vérification quantitative. Elle constitue le second axe abordé dans ce mémoire, aux côtés de la résolution paramétrée par la structure de la formule.

1.2 Approche structurelle et décomposition arborescente

Plusieurs familles d’algorithmes ont émergé pour résoudre #SAT et MaxSAT : solveurs généraux fondés sur DPLL et CDCL, outils SMT comme Z3 [9], solveurs de comptage avec cache de composantes comme sharpSAT [7], ou encore la compilation de connaissances déjà évoquée. Ce travail combine deux perspectives complémentaires : la *complexité paramétrée structurelle*, qui exploite la structure du graphe d’incidence pour résoudre #SAT et MaxSAT en temps polynomial dès que le paramètre est borné ; et la *compilation de connaissances*, qui transforme cette même structure en un objet polynomialement interrogeable.

L’idée fondatrice est que la difficulté d’une instance ne dépend pas uniquement de sa taille $n + m$ (variables et clauses), mais de paramètres structurels liés à la façon dont variables et clauses interagissent. Si ces paramètres sont petits, des algorithmes par *programmation dynamique sur décomposition arborescente* résolvent #SAT et MaxSAT en temps polynomial en $n + m$, l’explosion exponentielle étant confinée à ce seul paramètre.

Parmi ces paramètres, la *treewidth* du graphe d’incidence de la formule est le plus étudié [13, 14]. Des résultats plus récents, fondés sur la *mim-width* [15] et la *ps-width* [20], étendent la tractabilité à des classes de formules strictement plus larges. L’algorithme de Sæther, Telle et Vatshelle [20] résout simultanément #SAT et MaxSAT en temps $O(k^3 \cdot m \cdot (m + n))$, où k est la *ps-width* de la décomposition. Il constitue le premier pilier de ce mémoire. Son extension par Bova, Capelli, Mengel et Slivovsky [17] en forme le second : ces auteurs l’instrumentent pour produire en parallèle un DAG d-DNNF, faisant ainsi le pont avec la compilation de connaissances.

1.3 Objectifs de ce travail

Ce mémoire de bachelor poursuit deux objectifs complémentaires.

Le premier est l’**implémentation complète** de l’algorithme de Sæther, Telle et Vatshelle [20] en langage C. Cette implémentation comprend le parseur de formules DIMACS CNF, trois stratégies de construction de l’arbre de décomposition (dont l’heuristique GreedyOrder décrite dans le papier), les trois procédures de l’algorithme (calcul bottom-up des PS-sets, propagation top-down, programmation dynamique), ainsi que les structures de données sous-jacentes (bitsets 64-bit, trie binaire, tables DP).

Le second objectif est l’**extension à la compilation de connaissances** fondée sur le résultat de Bova, Capelli, Mengel et Slivovsky [17]. Leur algorithme s’obtient comme une instrumentation de la programmation dynamique précédente et produit, à coût marginal, un DAG d-DNNF de taille polynomiale en la *ps-width* et la taille de la formule. À notre connaissance, aucune implémentation publique ne suit aujourd’hui cette voie de compilation pilotée par la *ps-width*. Sur le DAG produit, l’ensemble des requêtes que la carte de compilation de Darwiche et Marquis [10] rend polynomiales sur d-DNNF est implémenté.

Une **évaluation empirique** accompagne ces deux volets, sur des instances synthétiques conçues pour contrôler la *ps-width*, des formules aléatoires k -SAT et des circuits de multiplication encodés par la transformation de Tseytin. Ces tests mesurent l’impact de la *ps-width* sur les performances et délimitent les régimes dans lesquels la décomposition structurelle apporte un avantage décisif.

2

État de l'art

Ce chapitre positionne notre travail dans la littérature. Nous présentons successivement les problèmes SAT, #SAT et MaxSAT, les principales familles d'algorithmes de résolution, les paramètres de décomposition structurelle qui gouvernent leur complexité, l'algorithme de Sæther, Telle et Vatshelle qui constitue le socle de notre implémentation, et la compilation de connaissances $CNF \rightarrow d\text{-DNNF}$ qui constitue le second axe de ce mémoire.

2.1 SAT, #SAT et MaxSAT

2.1.1 Le problème SAT

Le problème de satisfaisabilité propositionnelle (SAT) demande, étant donnée une formule F en forme normale conjonctive (CNF), s'il existe une affectation des variables qui la satisfait. Rappelons qu'une formule CNF est une conjonction de clauses, chaque clause étant une disjonction de littéraux (variable ou sa négation). Une affectation satisfaisante est appelée *modèle* de F .

SAT fut le premier problème formellement démontré NP-complet par Cook [1], puis Karp en étendit l'applicabilité en montrant que de nombreux problèmes combinatoires classiques s'y réduisent en temps polynomial [2]. Malgré cette intractabilité théorique, les solveurs industriels modernes traitent couramment des instances de plusieurs millions de variables et clauses, grâce à l'algorithme CDCL [4, 5] décrit à la section 2.2.1.

2.1.2 #SAT : comptage de modèles

Le problème #SAT (*model counting*) demande non plus si un modèle existe, mais *combien* il en existe. Ce problème est #P-complet [3], une classe de complexité qui capture les problèmes de comptage associés aux décisions NP. Valiant a notamment montré que même sur des fragments syntaxiques pour lesquels la décision est facile (2-CNF, Horn), le comptage reste #P-complet.

#SAT intervient naturellement dans l'inférence probabiliste sur des réseaux bayésiens [10], le model checking quantitatif, et l'analyse de fiabilité de systèmes. Sa résolution exacte exige d'explorer l'intégralité de l'espace de solutions, ce qui le distingue fondamentalement du problème de décision.

2.1.3 MaxSAT

Le problème MaxSAT cherche une affectation qui maximise le nombre de clauses satisfaites, même si la formule est globalement insatisfaisable. Il est NP-difficile et modélise des problèmes d'optimisation à contraintes souples - ordonnancement, configuration, bioinformatique. Des variantes importantes incluent le *Partial MaxSAT* (clauses dures obligatoires + clauses souples à maximiser) et le *Weighted MaxSAT* (poids par clause).

2.2 Algorithmes de résolution

2.2.1 DPLL et CDCL

L'algorithme DPLL (Davis-Putnam-Logemann-Loveland) constitue le fondement historique de la résolution SAT : il parcourt en profondeur l'espace des affectations partielles en s'appuyant sur deux règles d'inférence, la *propagation unitaire* (BCP), qui force la valeur d'un littéral dès qu'il est le dernier non falsifié d'une clause, et la *règle du littéral pur*. L'architecture CDCL (Conflict-Driven Clause Learning), popularisée par zChaff [4] puis MiniSAT [5], étend DPLL par l'*apprentissage de clauses* : à chaque conflit, le solveur extrait une nouvelle clause résumant la cause de l'échec, et peut ensuite effectuer un *backjumping* non chronologique. Couplée à des heuristiques de branchement orientées par l'activité des conflits (notamment VSIDS [4]), CDCL est aujourd'hui le standard industriel et traite couramment des instances de plusieurs millions de variables.

2.2.2 Extensions pour #SAT et MaxSAT

Pour le comptage de modèles, les solveurs étendent DPLL par le *cache de composantes* : lorsque l'affectation partielle déconnecte le graphe d'incidence, les comptes des composantes disjointes sont calculés indépendamment puis multipliés. Ce mécanisme, popularisé par *sharpSAT* [7], est combiné dans D4 [8] avec la compilation vers d-DNNF, rendant les requêtes ultérieures polynomiales. Pour la maximisation, les solveurs MaxSAT modernes (*MaxHS*, *RC2*, *Open-WBO*) abandonnent le *branch-and-bound* traditionnel au profit d'approches *core-guided*, qui résolvent itérativement des noyaux insatisfaisables au lieu d'explorer naïvement l'espace des affectations.

2.2.3 SMT et compilation de connaissances

Les solveurs SMT (*satisfiability modulo theories*), dont *Z3* [9], étendent CDCL en intégrant des théories arithmétiques via l'architecture DPLL(T) ; sur des instances purement propositionnelles, les solveurs SAT dédiés restent toutefois plus rapides. Une approche alternative, la *compilation de connaissances*, transforme hors-ligne la formule en une représentation polynomialement interrogeable ; elle fait l'objet de la section 2.5, où elle est articulée avec l'algorithme de Sæther via l'extension de Bova et al. [17].

2.3 Décompositions arborescentes et paramètres de largeur

Pour les formules issues du monde réel, la difficulté pratique dépend moins de la taille brute que de la structure de connectivité du graphe d'incidence. Plusieurs paramètres ont été proposés pour quantifier cette structure et fournir des algorithmes de programmation dynamique tractables quand le paramètre est borné.

Le plus étudié est la *treewidth*, introduite par Robertson et Seymour dans leur théorie des mineurs de graphes : on représente le graphe par une famille de petits sous-ensembles de sommets

(*sacs*) connectés en arbre, et la largeur est la taille du plus grand sac. Samer et Szeider [13] puis Slivovsky et Szeider [14] ont obtenu des algorithmes de DP pour #SAT paramétrés par la treewidth du graphe d'incidence. Toutefois, la treewidth n'est bornée que sur des graphes globalement clairsemés, ce qui en limite la portée sur les classes denses.

Pour dépasser cette limitation, des paramètres plus généraux ont été introduits, notamment la *mim-width* (*maximum induced matching width*) de Vatshelle [15], qui reste bornée sur des classes à treewidth non bornée (graphes d'intervalles ou de permutation, par exemple). En contrepartie, calculer la *mim-width* est intractable : Bergougnoux, Bonnet et Duron [16] ont récemment montré que le problème est paraNP-complet, ce qui impose le recours à des heuristiques dans toute mise en œuvre pratique.

2.4 PS-width et l'algorithme de Sæther-Telle-Vatshelle

Sæther, Telle et Vatshelle [20] introduisent un paramètre défini directement sur les formules CNF, la *ps-width*, fondé sur une *branch decomposition* de F : un arbre binaire dont les feuilles sont en bijection avec $\text{var}(F) \cup \text{cla}(F)$. À chaque nœud v de l'arbre, on observe les ensembles de clauses extérieures satisfaisables par les affectations partielles locales à v ; la *ps-width* de la décomposition est la cardinalité maximale de ces ensembles, et la *ps-width* de F le minimum sur toutes les décompositions possibles.

Ce paramètre domine la *mim-width* du graphe d'incidence : si la *mim-width* est bornée et les clauses de taille bornée, la *ps-width* est polynomiale en le nombre de clauses. Or une *mim-width* bornée n'implique pas une treewidth bornée ; la *ps-width* étend donc strictement les classes de formules tractables couvertes par les algorithmes paramétrés par la treewidth.

Les auteurs donnent un algorithme par programmation dynamique en trois procédures séquentielles le long de la décomposition (calcul bottom-up des PS-sets, propagation top-down, puis remplissage d'une table d'états combinant les enfants), résolvant simultanément #SAT et MaxSAT en $O(k^3 \cdot m \cdot (m + n))$, où k est la *ps-width*. Pour une décomposition linéaire (chaque nœud interne a au moins une feuille pour enfant), la complexité chute à $O(k^2 \cdot m \cdot (m + n))$. La construction détaillée des trois procédures, ainsi que la définition formelle de la *ps-width* et de ses relations avec les autres paramètres de largeur, fait l'objet du chapitre 3 de ce rapport.

Comme minimiser la *ps-width* est intractable en général, les auteurs proposent l'heuristique gloutonne *GreedyOrder*, qui construit un ordre linéaire des sommets du graphe d'incidence en privilégiant à chaque étape ceux qui ont le plus grand nombre de voisins déjà choisis. Pour valider l'algorithme, ils introduisent trois familles d'instances synthétiques (*Type 1*, graphes bipartis d'intervalles ; *Type 2*, mêmes structures avec clauses courtes ; *Type 3*, fonctions XOR cycliques formant des graphes d'arcs circulaires) sur lesquelles le DP paramétré par la *ps-width* surpasse nettement les solveurs CDCL [20].

2.5 Compilation de connaissances CNF $\rightarrow$ d-DNNF

La *compilation de connaissances* (*knowledge compilation*, KC) transforme une formule CNF hors-ligne en une représentation structurée, typiquement un DAG, sur laquelle certaines requêtes (comptage, conditionnement, projection partielle) deviennent polynomiales. Le coût de compilation est amorti dès qu'une même base est interrogée de multiples fois, et les progrès de la décennie ont placé la compilation au centre de l'inférence probabiliste exacte et de la vérification quantitative. Cette section résume la hiérarchie des langages cibles, les compilateurs contemporains, puis détaille le résultat de Bova, Capelli, Mengel et Slivovsky (BCMS 2016) qui constitue le fondement théorique de l'extension KC de notre implémentation.

2.5.1 Langages cibles

Darwiche et Marquis [10] ont dressé la *carte de compilation* qui classe les représentations selon deux axes : la *succinctitude* (taille relative) et la *tractabilité* des requêtes supportées en temps polynomial. La hiérarchie pertinente pour notre travail va de la NNF (*Negation Normal Form* : DAG dont les feuilles sont des littéraux et les nœuds internes des ET et OU) à la DNNF (*Decomposable NNF* : enfants de tout ET à portées disjointes), puis au **d-DNNF** (*Deterministic DNNF* : enfants de tout OU à ensembles de modèles disjoints). Cette dernière propriété rend le comptage #SAT polynomial par un parcours qui remplace OU par somme et ET par produit, et fait du d-DNNF le langage cible de notre implémentation. D'autres variantes notables existent, notamment le SDD [11] (d-DNNF contraint par un *utree*) et l'OBDD [12] (précurseur historique), plus restrictives mais non utilisées dans ce travail.

2.5.2 Requêtes tractables sur d-DNNF

L'intérêt pratique d'un langage cible se mesure aux requêtes qu'il admet en temps polynomial dans la taille de la représentation. Darwiche et Marquis [10] établissent que la classe d-DNNF rend polynomiales six requêtes standard : **CO** (*consistency*, F est-elle satisfaisable ?), **VA** (*validity*, F est-elle une tautologie ?), **CE** (*clausal entailment*, F implique-t-elle une clause donnée ?), **IM** (*implicant check*, un terme est-il un implicant de F ?), **CT** (*model counting*, le #SAT) et **ME** (*model enumeration*, énumération des modèles avec un délai polynomial entre modèles consécutifs).

Cette tractabilité tient à la combinaison de deux propriétés caractéristiques du langage, la *décomposabilité* (sous-arbres disjoints sous chaque ET) et le *déterminisme* (sous-arbres mutuellement exclusifs sous chaque OU) ; une troisième propriété, la *lissité*, est requise spécifiquement pour CT et VA et peut s'obtenir en temps polynomial. Le DAG produit par la construction BCMS 2016 (section 2.5.3) est lisse par construction, et ces six requêtes sont donc directement disponibles sur le résultat compilé. Leur algorithmique précise sur d-DNNF fait l'objet du chapitre 3.

2.5.3 Compilation explicite pilotée par la ps-width : BCMS 2016

Bova, Capelli, Mengel et Slivovsky [17] prolongent l'algorithme de Sæther, Telle et Vatschelle en montrant que toute formule CNF F dotée d'une branch decomposition de ps-width k peut être compilée en un d-DNNF structuré de taille

$$|D| \leq 7 \cdot k^3 \cdot (n + m),$$

en temps polynomial en $n + m$. L'algorithme s'obtient comme une *instrumentation* de la programmation dynamique de Sæther : les produits et sommes du DP deviennent respectivement des ET et OU dans le DAG, de sorte que parcourir ce dernier en substituant $\vee$ par $+$ et $\wedge$ par $\times$ redonne exactement le nombre de modèles calculé par la DP. Le résultat est une voie de compilation paramétrée polynomiale en $n + m$ dès que la ps-width est bornée, sans recourir à la recherche DPLL. La construction détaillée et les invariants structurels du DAG ainsi produit relèvent du cadre théorique (chapitre 3).

2.5.4 Évaluation et vérification

L'évaluation empirique des compilateurs est fédérée par la *Model Counting Competition* (MCC) [19], qui couvre depuis 2020 des instances issues de réseaux bayésiens, de circuits ISCAS et de modèles de configuration logicielle. Plus récemment, le framework *CPOG* de Bryant et al. [18] introduit la vérification formelle des compilateurs, en produisant parallèlement au DAG d-DNNF une preuve certifiable mécaniquement par l'assistant Lean 4 de son équivalence avec la CNF source.

3

Cadre théorique

4

Implémentation

5

Résultats expérimentaux

6

Discussion

7

Conclusion

A

Annexes

Bibliographie

Bibliographie

- [1] S. A. Cook. The complexity of theorem-proving procedures. In *Proc. 3rd Annual ACM Symposium on Theory of Computing (STOC)*, pp. 151–158, 1971.
- [2] R. M. Karp. Reducibility among combinatorial problems. In R. E. Miller and J. W. Thatcher (eds.), *Complexity of Computer Computations*, pp. 85–103. Plenum Press, 1972.
- [3] L. G. Valiant. The complexity of enumeration and reliability problems. *SIAM Journal on Computing*, 8(3) :410–421, 1979.
- [4] M. W. Moskewicz, C. F. Madigan, Y. Zhao, L. Zhang, and S. Malik. Chaff : Engineering an efficient SAT solver. In *Proc. 38th Design Automation Conference (DAC)*, pp. 530–535, 2001.
- [5] N. Eén and N. Sörensson. An extensible SAT-solver. In *Proc. 6th International Conference on Theory and Applications of Satisfiability Testing (SAT)*, LNCS 2919, pp. 502–518. Springer, 2003.
- [6] J. P. Marques-Silva and K. A. Sakallah. GRASP : A search algorithm for propositional satisfiability. *IEEE Transactions on Computers*, 48(5) :506–521, 1999.
- [7] M. Thurley. sharpSAT – Counting models with advanced component caching and implicit BCP. In *Proc. 9th International Conference on Theory and Applications of Satisfiability Testing (SAT)*, LNCS 4121, pp. 424–429. Springer, 2006.
- [8] J.-M. Lagniez and P. Marquis. An improved decision-DNNF compiler. In *Proc. 26th International Joint Conference on Artificial Intelligence (IJCAI)*, pp. 667–673, 2017.
- [9] L. de Moura and N. Bjørner. Z3 : An efficient SMT solver. In *Proc. 14th International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, LNCS 4963, pp. 337–340. Springer, 2008.
- [10] A. Darwiche and P. Marquis. A knowledge compilation map. *Journal of Artificial Intelligence Research (JAIR)*, 17 :229–264, 2002.
- [11] A. Darwiche. SDD : A new canonical representation of propositional knowledge bases. In *Proc. 22nd International Joint Conference on Artificial Intelligence (IJCAI)*, pp. 819–826, 2011.
- [12] R. E. Bryant. Graph-based algorithms for Boolean function manipulation. *IEEE Transactions on Computers*, C-35(8) :677–691, 1986.
- [13] M. Samer and S. Szeider. Algorithms for propositional model counting. *Journal of Discrete Algorithms*, 8(1) :50–64, 2010.
- [14] F. Slivovsky and S. Szeider. A faster algorithm for propositional model counting parameterized by incidence treewidth. In *Proc. 23rd International Conference on Theory and Applications of Satisfiability Testing (SAT)*, LNCS 12178, pp. 267–276. Springer, 2020.
- [15] M. Vatshelle. *New Width Parameters of Graphs*. PhD thesis, University of Bergen, 2012.
- [16] B. Bergougnoux, É. Bonnet, and J. Duron. Mim-width is paraNP-complete. In *Proc. 52nd International Colloquium on Automata, Languages, and Programming (ICALP 2025)*, LIPIcs, 2025. DOI : 10.4230/LIPIcs.ICALP.2025.25.

-
- [17] S. Bova, F. Capelli, S. Mengel, and F. Slivovsky. On compiling CNFs into structured deterministic DNNFs. In *Proc. 19th International Conference on Theory and Applications of Satisfiability Testing (SAT)*, LNCS 9710, pp. 199–214. Springer, 2016.
 - [18] R. E. Bryant, W. Nawrocki, J. Avigad, and M. J. H. Heule. Certified knowledge compilation with application to formally verified model counting. In *Proc. 26th International Conference on Theory and Applications of Satisfiability Testing (SAT)*, LIPIcs, 2023. DOI : 10.4230/LIPIcs.SAT.2023.6.
 - [19] J. Fichte and M. Hecher. The model counting competitions 2021-2023. *arXiv preprint arXiv :2504.13842*, 2025.
 - [20] S. H. Sæther, J. A. Telle, and M. Vatshelle. Solving #SAT and MaxSAT by dynamic programming. *Journal of Artificial Intelligence Research (JAIR)*, 54 :59–82, 2015.

Déclaration d'utilisation de l'IA